

Submission Cover Sheet

Course Details

Title: Big Data Management Systems

Code: DAT5004 S2_24

Term: Spring 2025

Assessment: **WRIT1**

Student Details

Tom Rowan ST20285213

Project Tutor

Dr. Sandeep Singh Sengar

Table of Contents

Course Details	1
Student Details	1
Project Tutor	1
Task 1 – PCOMMS Inc. Application Design.....	3
Task 1.1	3
Task 1.2	4
Task 1.3	7
Task 1.4	8
Task 2 – Reflection on PRAC1	9
Task One – Website Scraping	9
Task Two – Cricketers Database	9
Task Three – MongoDB Cluster	9
References	11

Task 1 – PCOMMS Inc. Application Design

Task 1.1

PCOMMS Inc. must utilise several types of data to run their business as proposed, each with different characteristics and storage needs.

Financial Records, Order Logs, Regulatory Reporting

This data is structured. Data integrity is essential for financial transactions, requiring ACID compliance. (Connolly 2015, pp 671). Furthermore, reports and compliance audits will be required. These needs would suit a relational database.

Inventory Data, Customer Reviews

This type of data is semi-structured, with incomplete or varied attributes between products and manufacturers. (Connolly 2015, pp 1130). It may be updated regularly. Customers may wish to leave reviews for products. A document-based database, which provides a flexible schema would be used.

Social Media Profiles, Login Data

This data is semi-structured, as each social media platform holds disparate profile data. PCOMMS will likely use this data to inform marketing, such as preferences and 'friend' lists. The website will also need to track user sessions. This indicates using a key-value store or document database.

Social Network 'Friend' Model

This data is taken directly from user social media profiles or can be inferred across many social media networks. This is complex, semi-structured data and its interconnected relationship driven nature implies the use of a graph database.

PCOMMS would use friendship data to target product marketing and promotions.

(210 words)

Task 1.2

Based on the data types identified above, this section details the recommended database technologies that PCOMMS should implement:

Financial Records, Order Logs, Regulatory Reporting

A relational database will be required for this structured data, where consistency and data integrity are more important than flexibility. Financial and customer order records have known fields (columns) that are generally all have complete data for each transaction, for example 'customer', 'cost', 'number ordered' and 'total price'. The maintenance of financial records and order logging will mean relatively few database operations per customer.

Suitable databases would include MySQL, PostgreSQL and Microsoft SQL Server. PostgreSQL is a good choice for this requirement for several reasons.

Firstly, PostgreSQL is available under the more permissive opensource MIT licence, rather than being owned by a commercial company like MySQL from Oracle and SQL Server from Microsoft. This will allow for lower costs to PCOMMS.

PostgreSQL focusses on reliability and data integrity and "has been ACID-compliant since 2001". It also scales horizontally using 'sharding'. (PostgreSQL, 2023) MySQL is designed to be suitable for heavy workloads, where speed is important, over transactional integrity. (GeeksforGeeks, 2024)

Relational databases have limitations when used with unstructured data. Missing data can sometimes be handled by using NULL values. However, the schema is fixed and cannot change dynamically when new fields are presented for storage.

Inventory Data, Customer Reviews

Product data making up the inventory is semi-structured as it could have different fields per product. For instance, compare a book and a television. A book record would have an ISBN, an author, and a publisher. The record for a television might have a screen size, a manufacturer and a list of input connectors, such as HDMI.

In addition, customer reviews may be added. These are structured data but can also be stored alongside the inventory.

This type of data is best stored in a document database. It would be difficult to design a relational database schema to fit all possible products, given the different fields described above. (MongoDB, 2024)

As potential customers browse the inventory, there may be many read operations from the database. Updates to the inventory may be frequent, but this would be vanishingly few in comparison with read operations. Even if customer reviews are included the majority will be read operations.

MongoDB is the most likely choice as it supports a schema-less design, uses JSON documents, and supports indexing for nested fields. This would allow maximum flexibility for disparate product datasheets from various manufacturers, while remaining sufficiently performant in a clustered deployment. (Khan, 2023)

The query language provided by MongoDB is not SQL, but rather its own flexible language which includes very powerful aggregation pipelines, making the database do the work in document selection. This allows for less complex frontend application code.

MongoDB has limitations around transactional consistency across replicas, but there will be few database writes, so small delays while updates propagate across cluster nodes can likely be mitigated by updating records at quieter times. (MongoDB, 2024)

Social Media Profiles, Login Session Data

Users of the PCOMMS website will log in using social media accounts. There are many different social media networks, all with their own definition of a 'profile record'. Most will have the concept of a username, while only some will have full name and a profile picture or avatar. Many will store other details such as telephone, email, workplace and other details. Most, but not all, will have lists of friends, followers or other connections.

As the social media profile is used as the login credential for the PCOMMS application, some information will also be stored such as currently logged in status, last login and other activity flags.

Therefore, the social media data will not fit well into a relational database as defining a schema would be challenging and inflexible.

A key-value store would be suitable for this data. Two possible contenders are Redis and Couchbase, both of which are efficient for handling user sessions. (Naik, 2024)

Couchbase would be recommended. It is more flexible as it provides both key-value and a document store, which allows for both the profile storage and session tracking use cases. It also uses in-memory caching and provides a SQL-like language called N1QL, whereas Redis does not support SQL queries. Finally, Redis is not truly open-source; Couchbase is open-source. (GeeksforGeeks, 2020)

While using Couchbase will balance speed and flexibility, it is slightly more complex to manage than Redis and may require more optimisation when deployed into production. (Brilian, 2025)

Social Network 'Friend' Model

Most social media platforms have the concept of a friend or follower list. PCOMMS intends to use these lists to build a database of relationships between users for marketing.

The database for storing these relationships will assist in normalising connections across social platforms and inferring connections between users. The best type of database for modelling interconnections and relationships is a graph database such as Neo4J.

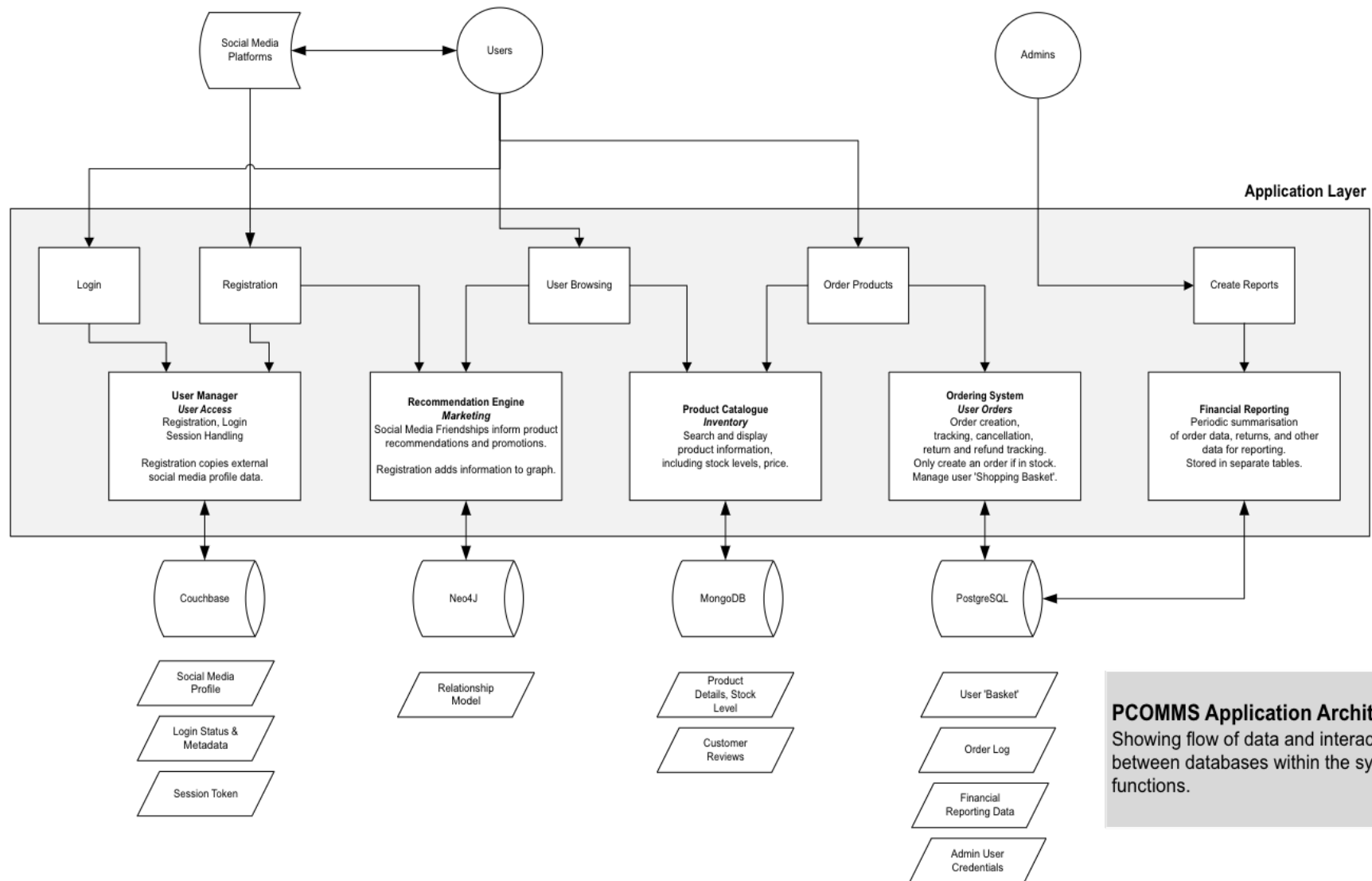
This type of data would be almost impossible to model in a relational database. "Social media networks are already graphs, so there's no point converting a graph into tables and then back again." (Neo4J, 2024)

Neo4J is optimised for modelling and searching relationships and can join disparate data to infer relationships. It is also often used to create recommendation engines that can represent user behaviour and suggest products based on recorded preferences obtained from social media profiles (FromDev 2023). This behaviour is perfect for PCOMMS needs.

Neo4J is a commercial database, and an enterprise licence may be cost-prohibitive. It is also recognised that learning to best utilise this kind of database is complex, and there may be a steep learning curve, impacting development and time to market for PCOMMS. (Mehta, 2024)

(945 words)

Task 1.3



Task 1.4

The application code acts as an intermediary between the various database components and transfers data. The application is responsible for session separation, such that users cannot access administration functions or each other's data.

Data Flows

- User login: Social media user profiles are stored in Couchbase. Session token data is also stored in Couchbase, in a key-value pair model.
- User browsing: Preference data from social profiles (Couchbase) is used where available to feed into a product recommendation engine displaying possible purchases (Neo4J).
- Ordering: The application checks MongoDB to ensure product availability before writing to the order log (PostgreSQL) and decrementing stock levels (MongoDB).
- Financial reporting: Order data is extracted periodically from PostgreSQL and stored in separate tables.
- Marketing: User connectivity data from social media is stored in Neo4j, which identifies influential users and targets their network for promotions. Preference data from profiles (Couchbase) is used where available. Neo4j in turn enriches user profiles.

Challenges

- Data Consistency: Ensuring consistency across databases may be complicated. The application must ensure transactional completion before confirming an order to a customer, for example.
- Latency: Multiple databases may introduce latency, so cache layers may be required, particularly in front of the product inventory (MongoDB) where there are large numbers of read operations.

(205 words)

Task 2 – Reflection on PRAC1

Task One – Website Scraping

For scraping movie data from the FilmCrave website, I used two python libraries: 'requests' and 'BeautifulSoup'. Both are simple to use and supported the task very well.

Each movie has its own page with more information than the Top 1000 listing summary pages, including a release date. I wrote functions that can scrape any film if given the URL for its page. This approach is slower, requiring more GET requests, but is more flexible. I scraped 57501 movies, taking approximately eight hours to complete and creating a huge dataset for analysis. If I were writing this function again, I would use asynchronous parallel requests.

The website uses Unicode encoding for non-English characters. Initially, I was using the 'response.**content**' method and had issues with incorrect characters in my results. The documentation for *requests* states that the 'response.**text**' method should be used for Unicode support. This fixed my issue, and I shared this fact with the other students.

Task Two – Cricketers Database

The Cricketers database was written in a single python file and provides a console-based application, with an SQLite backend. SQLite is suitable for this task as a relational database, as the dataset is well structured.

I wrote functions to handle the menu, perform SQL queries and display the results. I am pleased with the display function, which automatically handles padding and colours the data to create a table.

I found a cricketing data website for sample data. This posed a challenge as the data is protected from dynamic scraping. I saved the HTML to a file and was able to scrape the data. I did not face any other challenges with this task, as I have worked with SQL queries before.

Task Three – MongoDB Cluster

The PyMongo library is provided by the MongoDB developer, so is a good choice for interacting with the MongoDB cluster. Alternatives do exist, but these are for specialised projects.

I had some challenges initially with understanding the query language used to create aggregation pipelines. I now find it to be a very powerful tool, with many possible operations. The database does the hard work of filtering documents so that the application can be lightweight.

I reworked the table display function from the Cricketers database to display generic data from MongoDB results.

I have a personal project in progress that I will rewrite to use MongoDB (from PostgreSQL) following this exposure to the technology.

(402 words)

References

1. **Brilian, F. (2025)** *Couchbase Performance Optimization: Tips and tricks for high scalability*. <https://medium.com/@firmanbrilian/couchbase-performance-optimization-tips-and-tricks-for-high-scalability-1fb6c32f1be4>, (Accessed: 20/05/2025)
2. **Connolly, T.M. and Begg, C.E. (2015)** *Database systems: A practical approach to design, implementation and management*. Harlow, Essex, England: Pearson Education Limited.
3. **FromDev (2023)** *10 Powerful use cases for Neo4J Graph Databases*. <https://www.fromdev.com/2023/05/10-powerful-use-cases-for-neo4j-graph-databases.html>, (Accessed: 20/05/2025)
4. **GeeksforGeeks (2020)** *Difference between Couchbase and Redis*. <https://www.geeksforgeeks.org/difference-between-couchbase-and-redis/>, (Accessed: 20/05/2025)
5. **GeeksforGeeks (2024)** *Difference between MySQL and PostgreSQL*. <https://www.geeksforgeeks.org/difference-between-mysql-and-postgresql>, (Accessed: 20/05/2025)
6. **Khan, S. (2023)** *MongoDB 101: An Introduction to MongoDB Features, Comparison with SQL, Use Cases, and Installation Guide*. <https://medium.com/@skhans/mongodb-101-an-introduction-to-mongodb-features-comparison-with-sql-use-cases-and-installation-a66c21111c8e>, (Accessed: 20/05/2025)
7. **Mehta, K. (2024)** *Exploring Graph Databases: NEO4J*. <https://dev.to/kartikmehta8/exploring-graph-databases-neo4j-286g>, (Accessed: 20/05/2025)
8. **MongoDB (2024)** *NoSQL vs SQL Databases*. <https://www.mongodb.com/resources/basics/databases/nosql-explained/nosql-vs-sql>, (Accessed: 20/05/2025)
9. **Naik, S. (2024)** *Best Databases for Social media Apps*, Olibr Blogs, 29 March. <https://olibr.com/blog/best-databases-for-social-media-apps/>, (Accessed: 20/05/2025)
10. **Neo4j (2024)** *Social Network Graph Database use cases*. <https://neo4j.com/use-cases/social-network/>, (Accessed: 20/05/2025)
11. **PostgreSQL (2023)** *PostgreSQL: About*. <https://www.postgresql.org/about/>, (Accessed: 20/05/2025)